

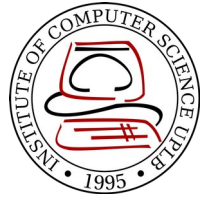
CMSC 137

Data Communications and Networking

Lecture Module
22 – Transport Layer



University of the Philippines
LOS BAÑOS



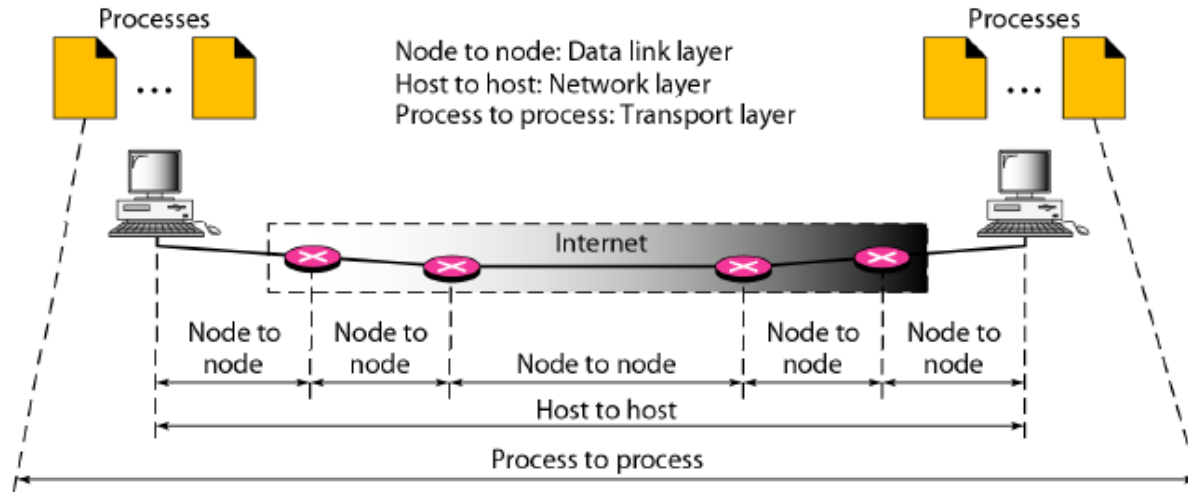
Module 22: Transport Layer

Rozano S. Maniaol
Lecturer

Introduction



The transport layer is responsible for the delivery of a message from one process to another

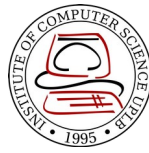


Client/Server Paradigm

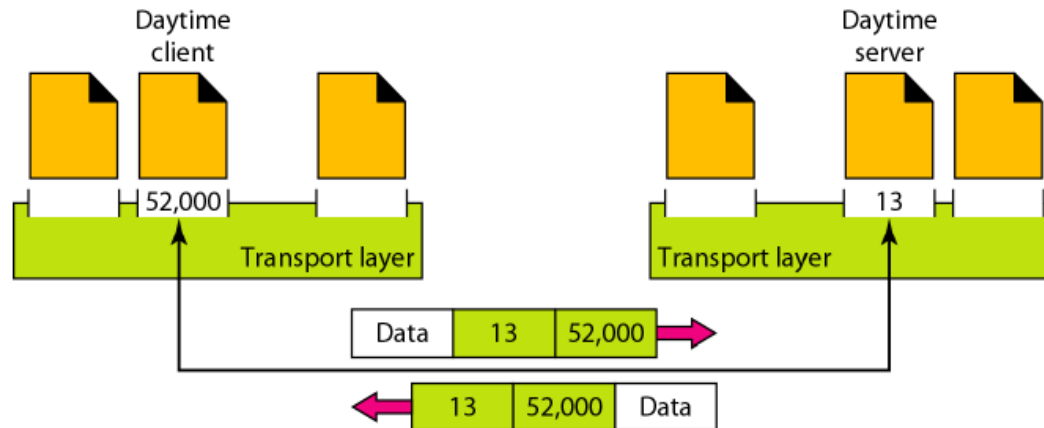


An application program on the local host, called the client, needs services from an application program on the remote host, called a server

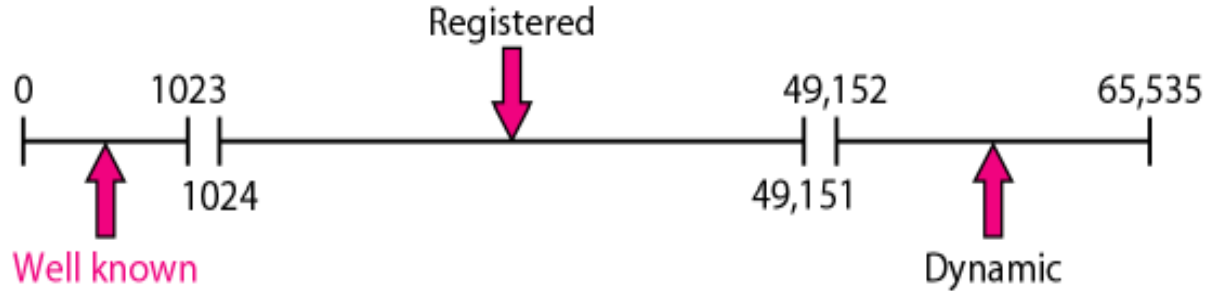
Port Number



- 16-bit integers between 0 and 65,535
- Client program defines itself with a port number (**ephemeral port number**), chosen randomly by the transport layer software running in the client host
- Servers use universal port numbers (**well-known port number**)



Port Number



- **Well-known ports** – ports ranging from 0 to 1023 and are assigned and controlled by the Internet Corporation for Assigned Names and Numbers (ICANN)
- **Registered ports** – ports ranging from 1024 to 49,151 and are not assigned or controlled by ICANN. They can only be registered to prevent duplication.
- **Dynamic (or private) ports** – ports ranging from 49,152 to 65,535 and are neither controlled nor registered. They can be used by any process

Port Number



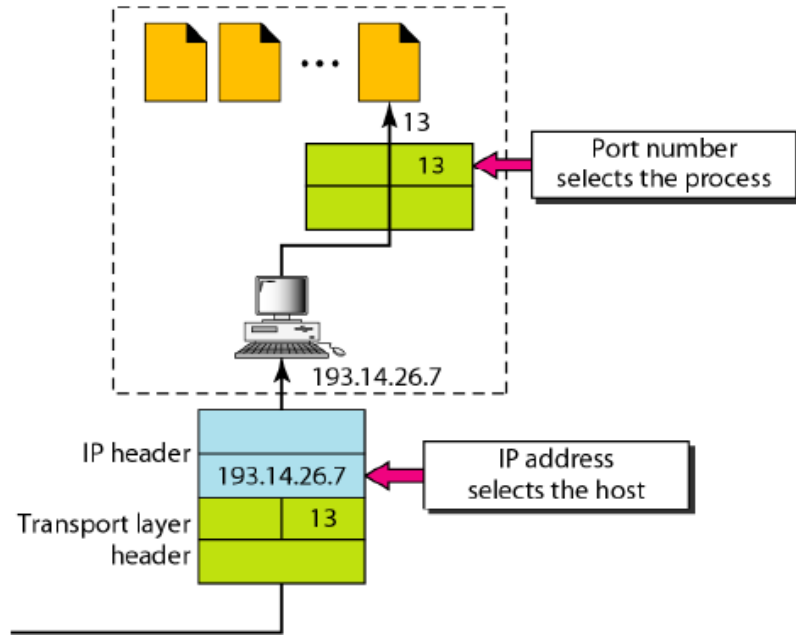
In UNIX, the well-known ports are stored in a file called `/etc/services`. Each line gives the name of the server and the well-know port number. Use `grep` utility to extract the line to get corresponding desired application

```
rsmaniaol@banaba:~$ grep http /etc/services
# Updated from http://www.iana.org/assignments/port-numbers and other
# sources like http://www.freebsd.org/cgi/cvsweb.cgi/src/etc/services .
http      80/tcp          www                # WorldWideWeb HTTP
https     443/tcp          # http protocol over TLS/SSL
http-alt  8080/tcp         webcache          # WWW caching service
http-alt  8080/udp
rsmaniaol@banaba:~$ grep ssh /etc/services
ssh       22/tcp           # SSH Remote Login Protocol
```

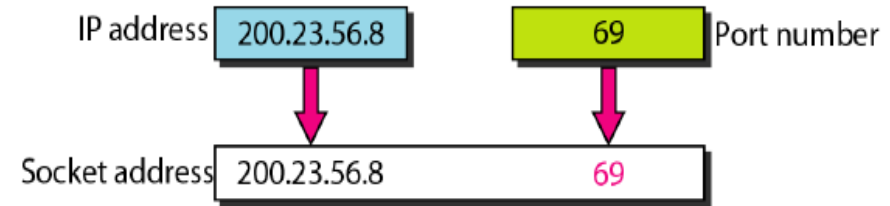
Socket Address



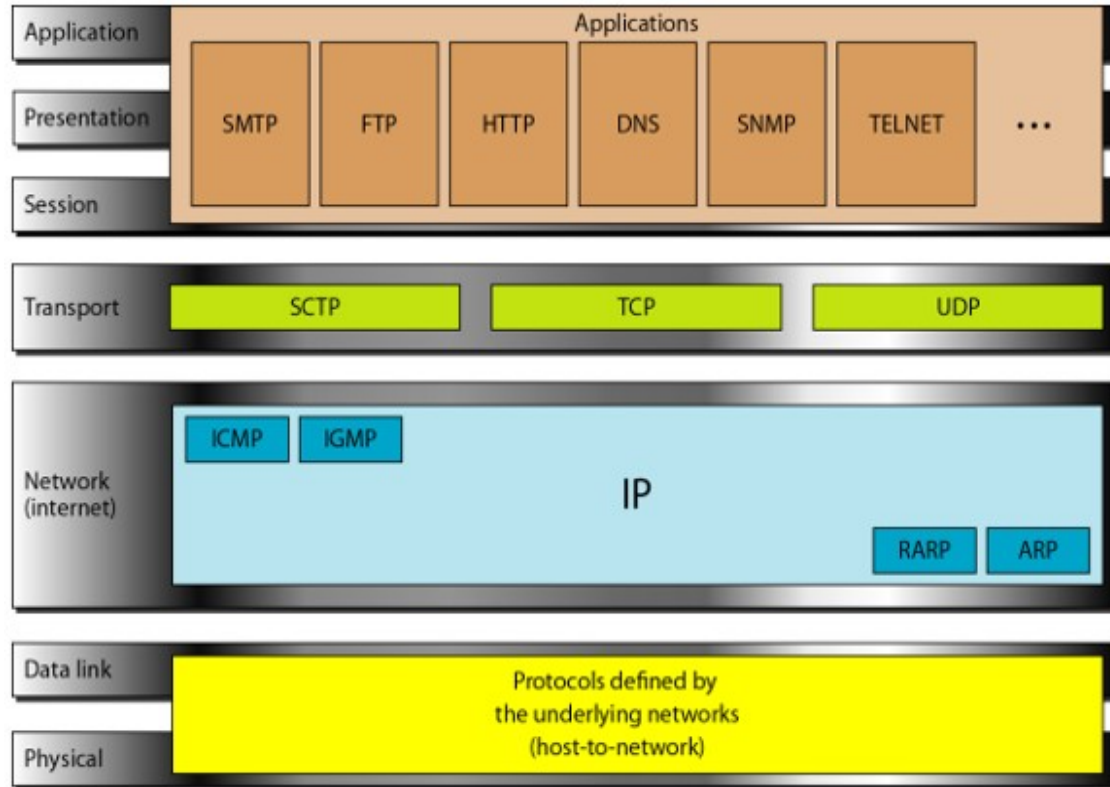
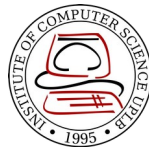
IP Address vs Port Numbers



Socket Address



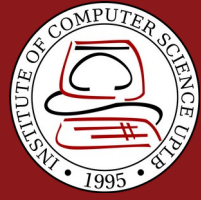
UDP, TCP, SCTP in TCP/IP Protocol Suite



Some well-known ports



Port	Protocol	UDP	TCP	SCTP	Description
7	Echo	✓	✓	✓	Echoes back a received datagram
9	Discard	✓	✓	✓	Discards any datagram that is received
11	Users	✓	✓	✓	Active users
13	Daytime	✓	✓	✓	Returns the date and the time
17	Quote	✓	✓	✓	Returns a quote of the day
19	Chargen	✓	✓	✓	Returns a string of characters
20	FTP-data		✓	✓	File Transfer Protocol
21	FTP-21		✓	✓	File Transfer Protocol
23	TELNET		✓	✓	Terminal Network
25	SMTP		✓	✓	Simple Mail Transfer Protocol
53	DNS	✓	✓	✓	Domain Name Service
67	DHCP	✓	✓	✓	Dynamic Host Configuration Protocol
69	TFTP	✓	✓	✓	Trivial File Transfer Protocol
80	HTTP		✓	✓	HyperText Transfer Protocol
111	RPC	✓	✓	✓	Remote Procedure Call
123	NTP	✓	✓	✓	Network Time Protocol
161	SNMP-server	✓			Simple Network Management Protocol
162	SNMP-client	✓			Simple Network Management Protocol



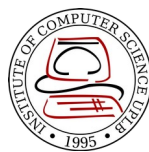
User Datagram Protocol

User Datagram Protocol (UDP)



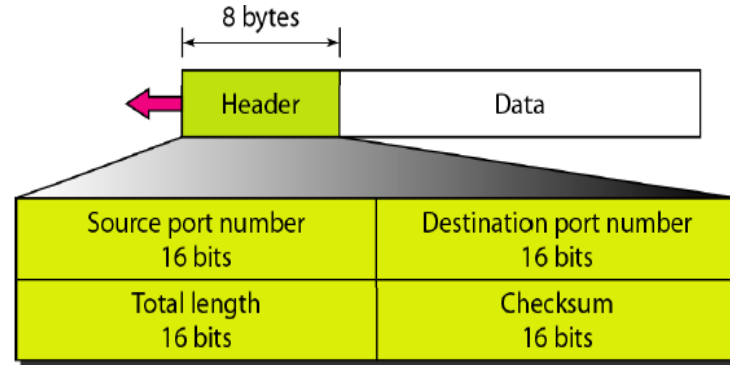
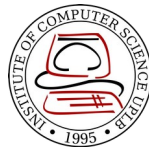
- Connectionless, unreliable transport protocol
- Does not add anything to IP except provide process-to-process communication
- Simple protocol using minimum overhead
- **RFC 768**

UDP



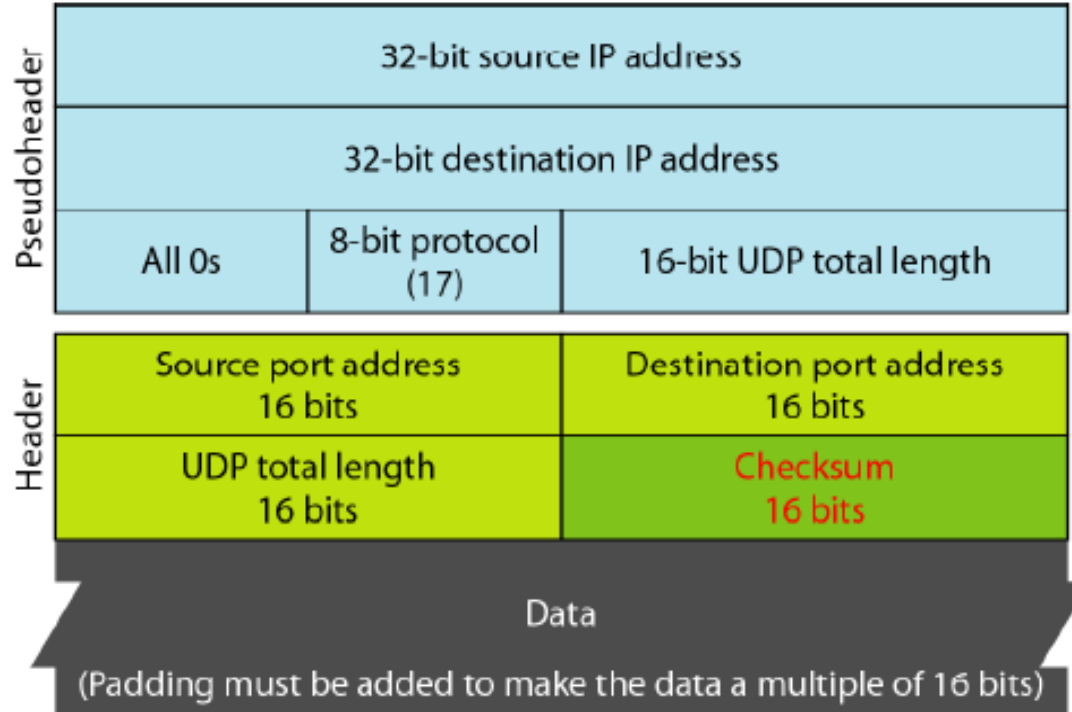
<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	BOOTPs	Server port to download bootstrap information
68	BOOTPc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

UDP Header

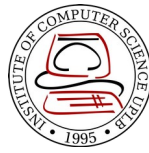


- Source port number – 16-bit long number used by the process running on the source host
- Destination port number - 16-bit long number used by the process running on the destination host
- Length – 16-bit field that defines the total length of the user datagram, header plus data; define up to 65,535 bytes
- Checksum – used to detect errors over the entire user datagram

Pseudoheader for checksum calculation



Question



- Given the content of a UDP header in hexadecimal format

CB84000D001C001C

- What is the source port number?

Source port number is the 1st 4 hexadecimal digits (CB84)₁₆, which means the source port is **52100**

- What is the destination port number?

Destination port number is the 2nd 4 hexadecimal digits (000D)₁₆, which means the destination port is **13**

- What is the total length of the user datagram?

3rd hexadecimal (001C)₁₆ define the length of the whole UDP packet as **28 bytes**

Question



- Given the content of a UDP header in hexadecimal format

CB84000D001C001C

- What is the length of the data?

Length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes

- Is the packet directed from a client to a server or vice versa?

Since the destination port number is a well-known port, the packet is from the **client to a server**

- What is the client process?

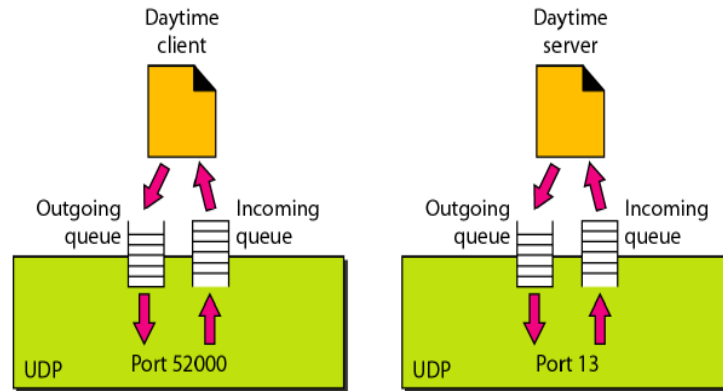
The client process is the **Daytime**

Queuing

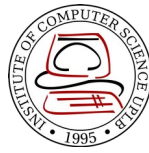


Some implementation create both an incoming and outgoing queue associated with each process; some only create an incoming queue

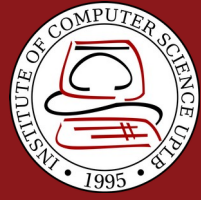
If no queue, UDP discards the user datagram and ask ICMP to send a port unreachable message to the server



Uses of UDP



- Suitable for a process that requires simple request-response communication
- Suitable for a process with internal flow and control mechanism, e.g. TFTP
- Suitable for multicasting
- Used for management processes such as SNMP
- Used for some routing protocols such as Routing Information Protocol
- Used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message

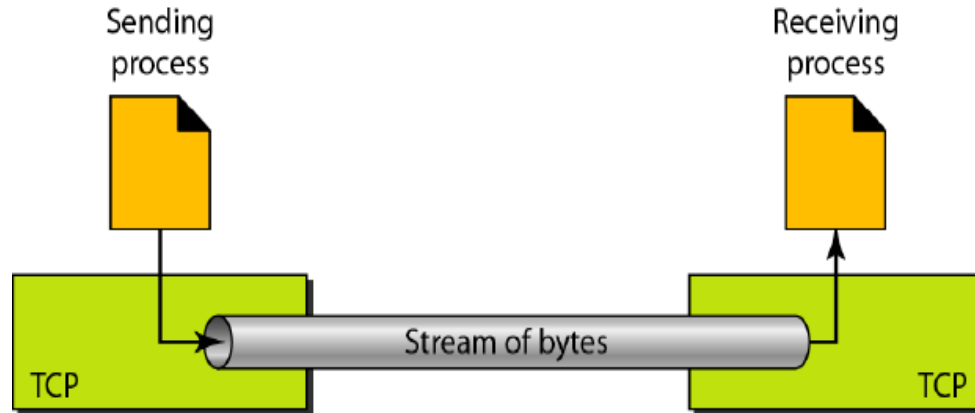


Transmission Control Protocol

Transmission Control Protocol (TCP)

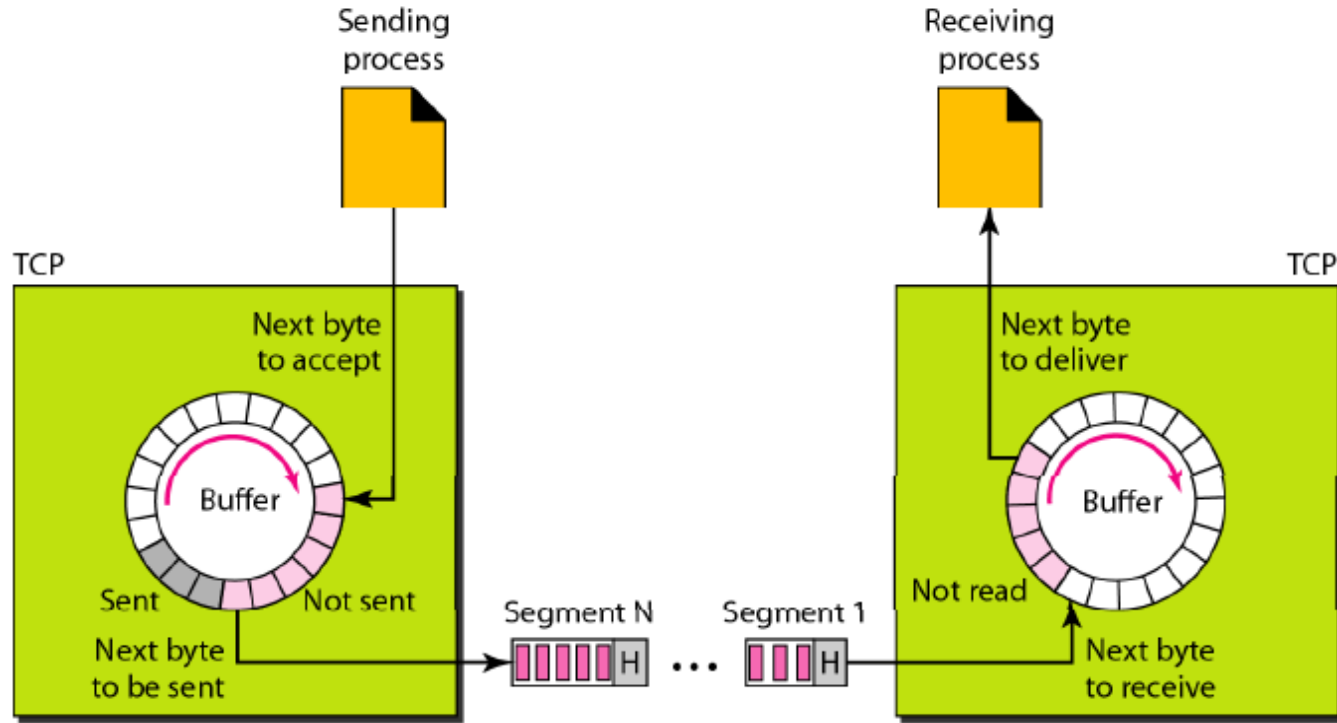


- Connection-oriented, reliable transport protocol
- Unlike UDP, it allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as stream of bytes
- RFC 793

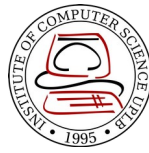


<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

Buffering

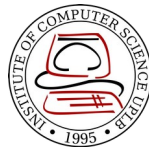


TCP Features



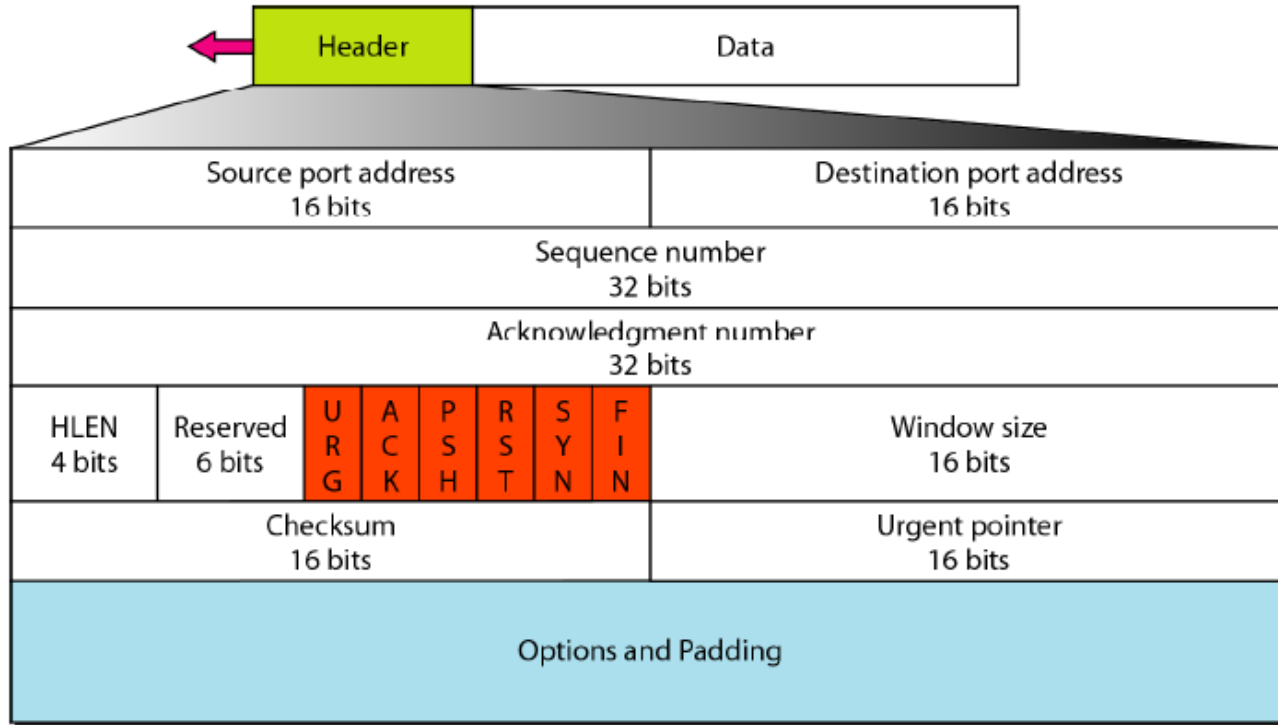
- **Numbering System**
 - **Byte number** – TCP numbers all data bytes that are transmitted in a connection. Numbering is independent in each direction and starts with a randomly generated number between 0 and $2^{32} - 1$
 - TCP assigns a **sequence number** to each segment that is being sent. The sequence number for each segment is the number of the first byte carried in that segment
 - When a connection is established, both parties can send and receive data at the same time. Each party uses an **acknowledgment number** to confirm the bytes it has received

TCP Features

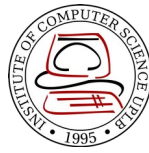


- **Flow control** – receiver of data controls the amount of data that are to be sent by the sender
- **Error control** – to provide reliable service, TCP implements an error control mechanism
- **Congestion control** – amount of data sent by the sender is not only controlled by the receiver, but is also determined by the level of congestion in the network

TCP Segment Format



TCP Segment Format

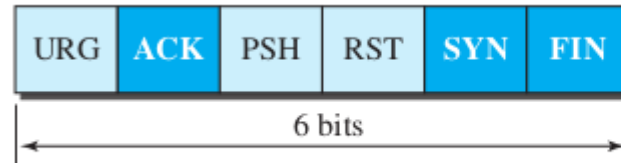


- **Source port address** – 16-bit field that defines the port number of the application program in the host that is sending the segment
- **Destination port address** – 16-bit field that defines the port number of the application program in the host that is receiving the segment
- **Sequence number** – 32-bit field that defines the number assigned to the first byte of data contained in the segment. During connection establishment, each party uses a random number generator to create an initial sequence number (ISN)

TCP Segment Format



- **Acknowledgment number** – 32-bit field that defines the byte number that the receiver of the segment is expecting to receive from the other party
- **Header length** – 4-bit field indicates the number of the 4-byte word in the TCP header
- **Reserved** – 6-bit field for future use
- **Control** – 6 different control bits or flags



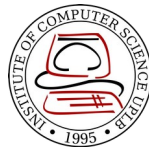
URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH : Request for push
RST : Reset the connection
SYN: Synchronize sequence numbers
FIN : Terminate the connection

TCP Segment Format



- **Window size** – 16-bit field that defines the size of the window, in bytes, that the other party must maintain. Normally referred to as the receiving window (rwnd) and has max value of 65,535 bytes
- **Checksum** – 16-bit field where the calculation is same as UDP. Inclusion of the checksum in TCP is mandatory while it is optional in UDP
- **Urgent pointer** – 16-bit field that defines the number that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment
- **Options** – Can be up to 40-bytes of optional information

TCP Connection

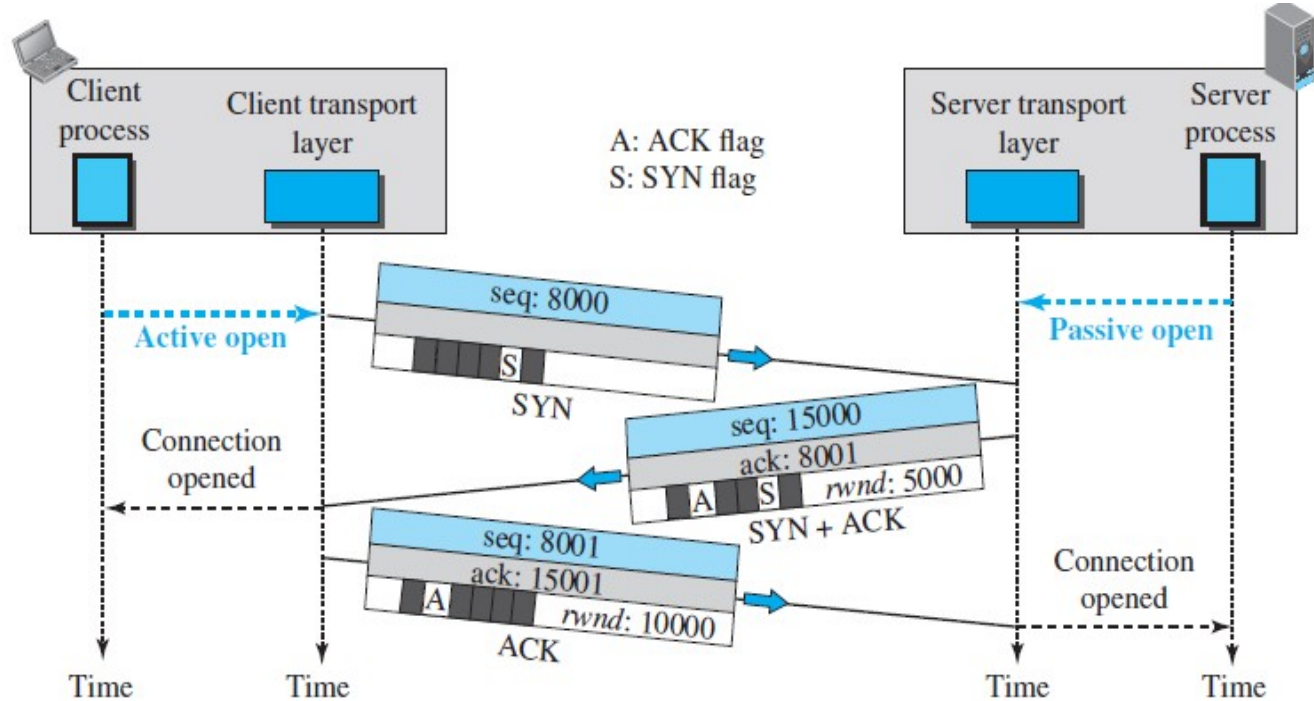


- TCP establishes a virtual path between the source and destination
- TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself
- It requires 3 phases: **connection establishment**, **data transfer**, and **connection termination**

Connection Establishment



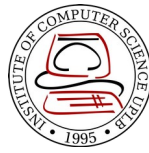
Three-way Handshaking



*A SYN+ACK segment cannot carry data, but it does consume one sequence number

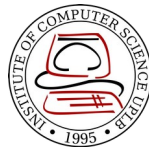
*An ACK segment, if carrying no data, consumes no sequence number

Connection Establishment



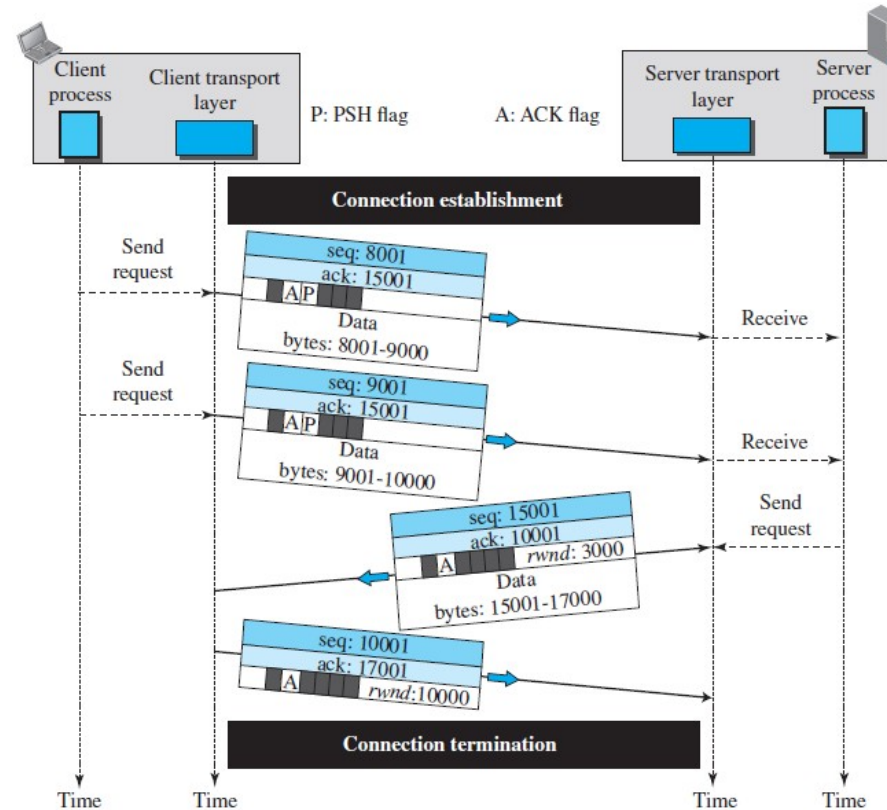
- Process starts with the server where it tells its TCP that it is ready to start a connection (**passive open**)
- A client that wishes to connect to an open server tells its TCP that it needs to be connected to that particular server (**active open**)
- A rare situation (**simultaneous open**) may occur when both processes issue an active open. Both TCPs transmit a SYN + ACK segment to each other, and one single connection is established between them

Connection Establishment

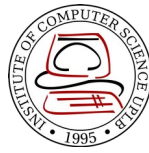


- Susceptible to the **SYN flooding attack**
 - Malicious attacker sends a large number of SYN segments to a server, pretending that each of them is coming from different clients by faking the source IP address in the datagrams
 - The server, assuming that the clients are issuing an active open, allocates the necessary resources and then sends the SYN + ACK segments to the fake clients which are lost

Data Transfer



Data Transfer

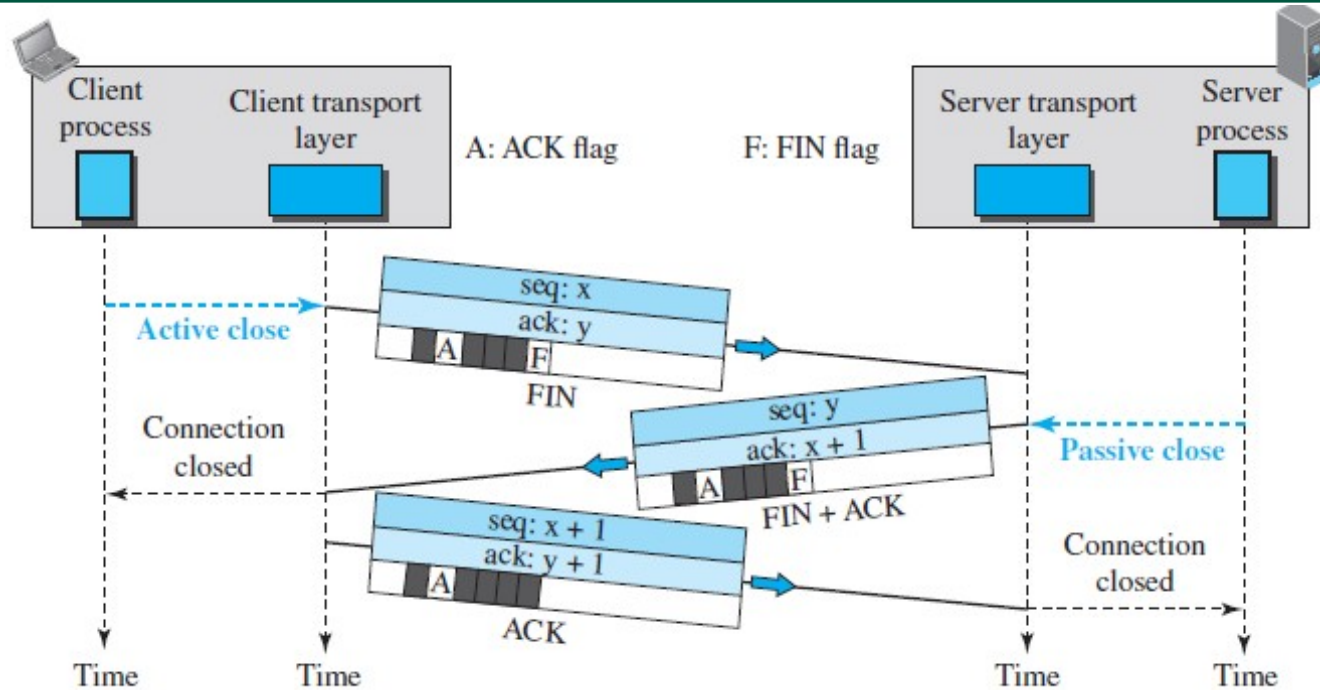


- After connection is established, bidirectional data transfer can take place. The client and server can both send data and acknowledgments
- **Pushing Data** – The sending TCP must not wait for the window to be filled, it must create a segment and send it immediately
- **Urgent Data** – the sending application program wants a piece of data to be read out of order by the receiving application program

Connection Termination



Three-way Handshaking

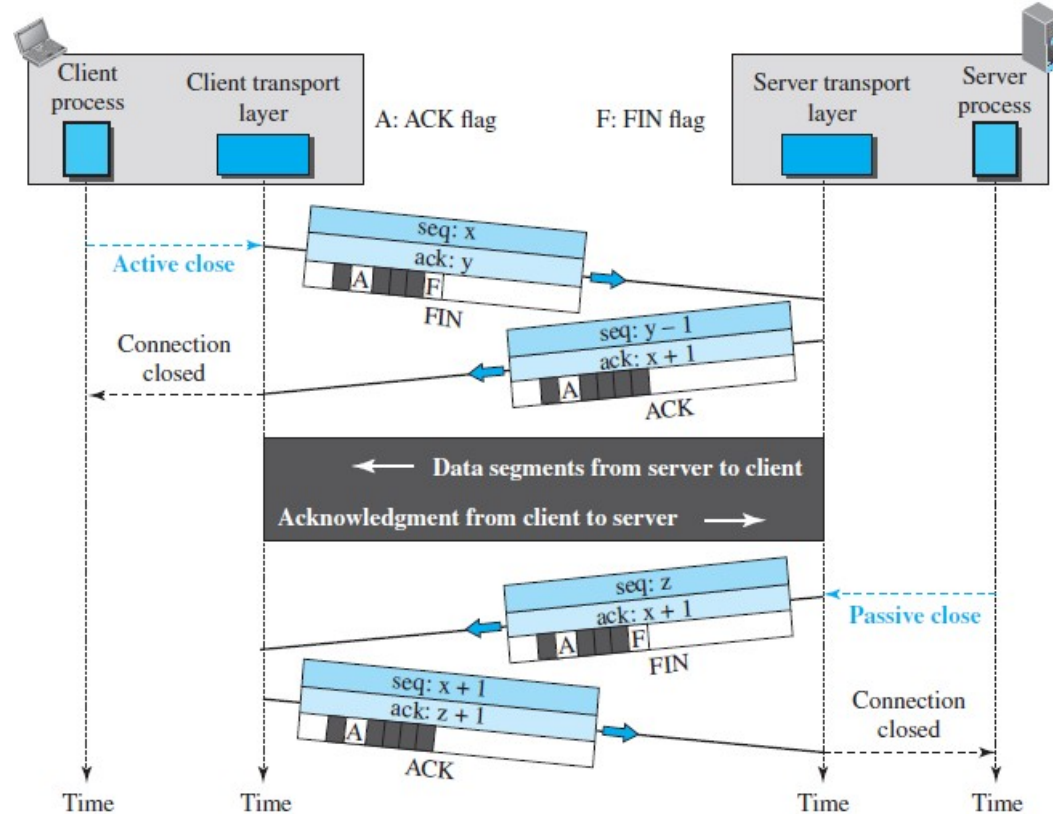


* The FIN segment consumes one sequence number if it does not carry data

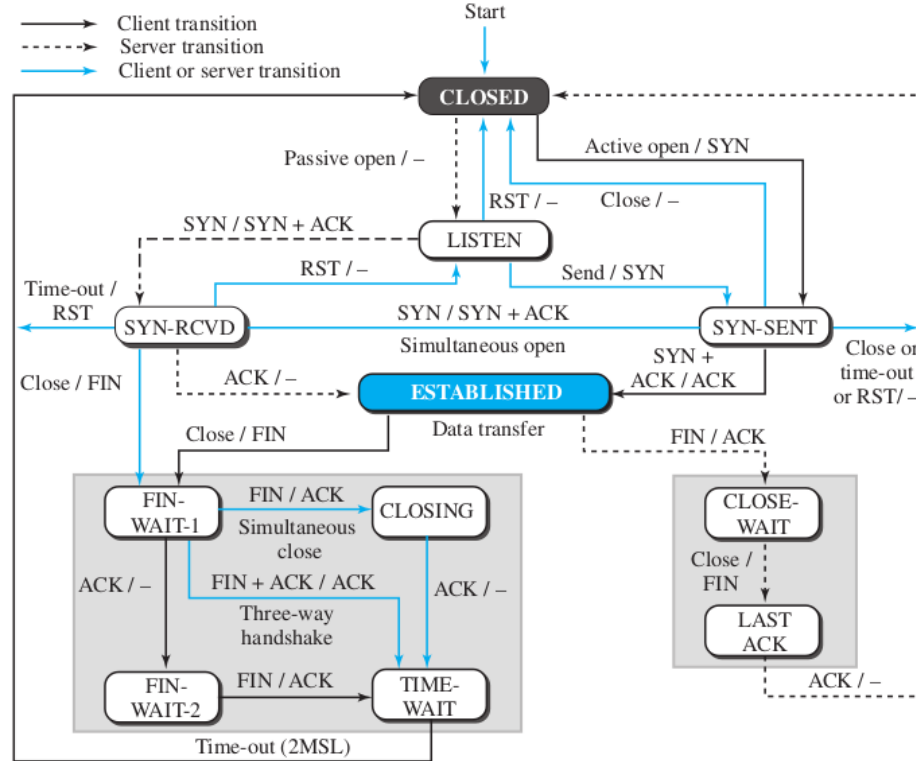
Connection Termination



Four-way Handshaking with half-close option



TCP State Transition Diagram



TCP Connection States



<i>State</i>	<i>Description</i>
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN + ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

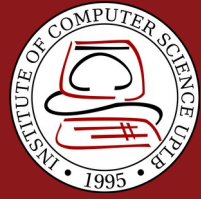
TCP Connection States



- Using the ss tool to view active TCP connections in Linux

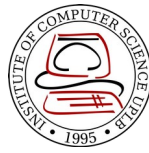
```
→ ss -atn
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	4096	0.0.0.0:3000	0.0.0.0:*	
LISTEN	0	32	192.168.1.123:53	0.0.0.0:*	
LISTEN	0	4096	127.0.0.1:53	0.0.0.0:*	
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	511	127.0.0.1:6463	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:631	0.0.0.0:*	
ESTAB	0	0	192.168.1.123:33486	142.251.221.35:443	
TIME-WAIT	0	0	192.168.1.123:40274	142.251.221.14:443	
ESTAB	32	0	192.168.1.123:54326	192.168.1.1:139	
ESTAB	0	0	192.168.1.123:34418	142.251.220.206:443	
ESTAB	0	0	192.168.1.123:34240	54.179.65.250:443	
ESTAB	0	0	192.168.1.123:48334	162.159.130.234:443	
ESTAB	0	0	192.168.1.123:54108	172.217.194.94:443	
ESTAB	0	0	192.168.1.123:47312	142.251.221.14:443	
TIME-WAIT	0	0	192.168.1.123:40448	142.251.220.138:443	
ESTAB	0	0	192.168.1.123:49828	142.251.220.131:443	
TIME-WAIT	0	0	192.168.1.123:50236	142.251.175.84:443	
ESTAB	0	0	192.168.1.123:43320	142.251.220.161:443	
ESTAB	0	0	192.168.1.123:34224	54.179.65.250:443	
ESTAB	0	0	192.168.1.123:45878	54.179.65.250:443	
ESTAB	0	0	192.168.1.123:34222	54.179.65.250:443	
ESTAB	0	0	192.168.1.123:37068	142.251.220.196:443	
ESTAB	0	0	192.168.1.123:34206	54.179.65.250:443	
ESTAB	0	0	192.168.1.123:37246	74.125.68.188:443	



Stream Control Transmission Protocol

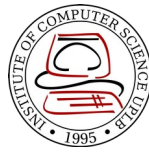
Stream Control Transmission Protocol



- New reliable, message-oriented transport layer protocol
- Preserves the message boundaries and at the same time detects lost data, duplicate data and out-of-order data
- It has also congestion control and flow control mechanisms
- **RFC 4960**

<i>Protocol</i>	<i>Port Number</i>	<i>Description</i>
IUA	9990	ISDN over IP
M2UA	2904	SS7 telephony signaling
M3UA	2905	SS7 telephony signaling
H.248	2945	Media gateway control
H.323	1718, 1719, 1720, 11720	IP telephony
SIP	5060	IP telephony

SCTP Services

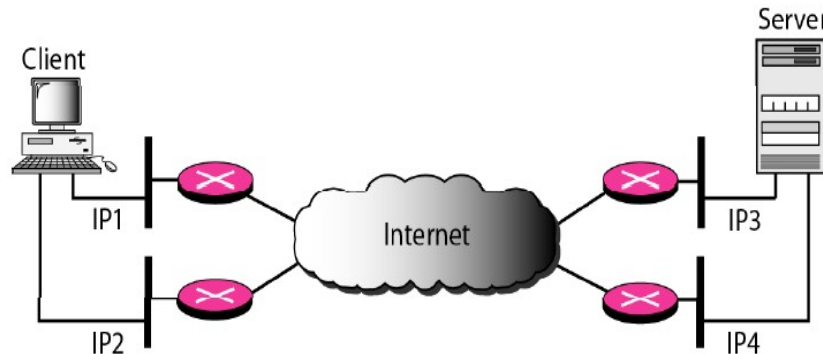


- Multiple streams
 - In TCP, each connection between client and server involves a single stream, a loss at any point in the stream blocks the delivery of the rest of the data
 - Allows multistream service in each connection (association in SCTP terminology)

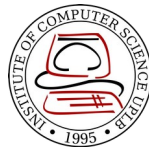
SCTP Services



- **Multihoming**
 - A TCP connection involves 1 source and 1 destination IP address, even if the sender or receiver is multihomed (connected to more than 1 physical address with multiple IP address), only one of these IP addresses per end can be used
 - Sending and receiving host can define multiple IP addresses in each association
 - When one path fails another interface can be used for data delivery without interruption

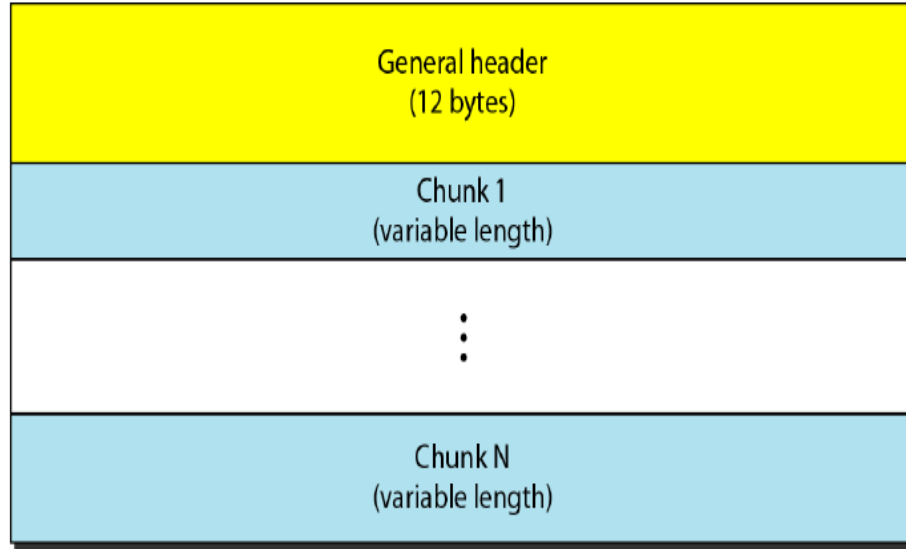


SCTP Feature



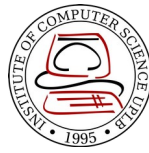
- **Transmission Sequence Number (TSN)**: unit of data in SCTP is a data chunk. Data transfer is controlled by numbering the data chunk using TSN
- **Stream Identifier (SI)** – each stream in SCTP needs to be identified
- **Stream Sequence Number (SSN)** – SCTP defines each data chunk in each stream. When a data chunk arrives at the destination, it is delivered to the appropriate stream and in the proper order
- **Packets** – data are carried as data chunks, control information is carried as control chunks. Several control chunks and data chunks can be packed together in a packet

SCTP Packet Format



- In an SCTP, control chunks come before data chunks

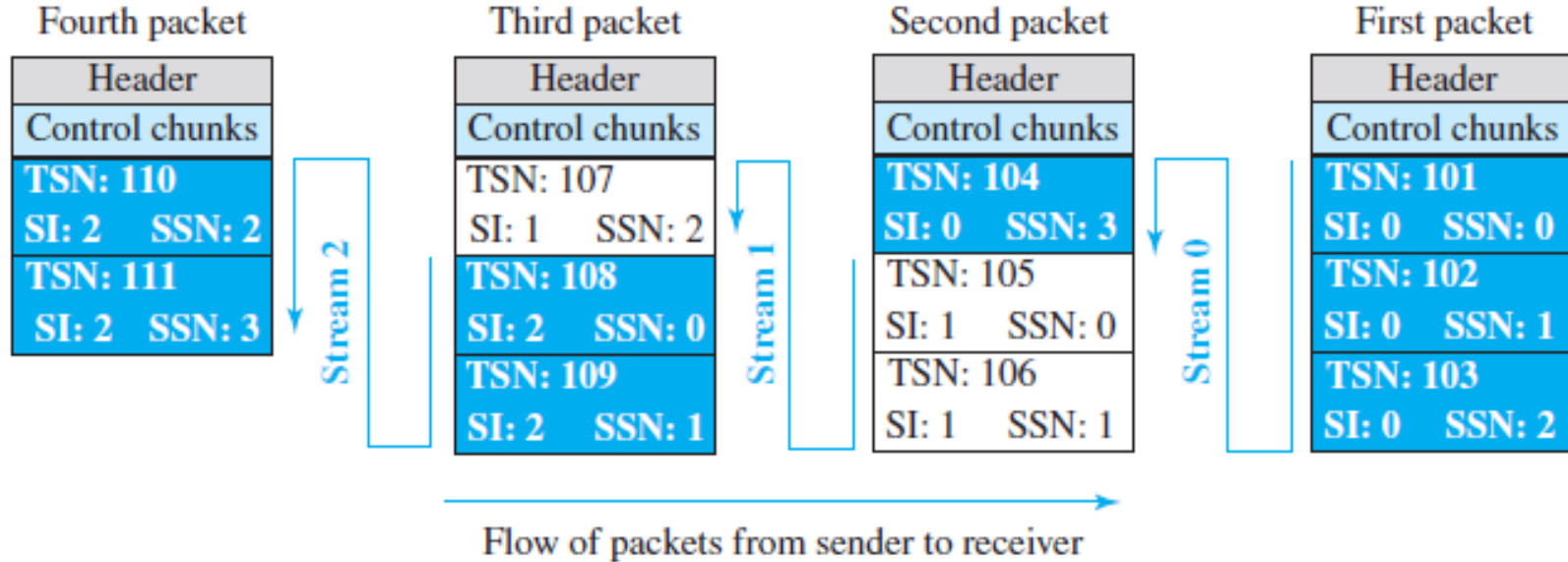
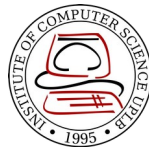
SCTP General Header



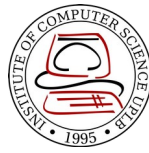
Source port address 16 bits	Destination port address 16 bits
Verification tag 32 bits	
Checksum 32 bits	

- **Source Port Address and Destination Address** – 16-bit field that defines the port numbers of the source and destination respectively
- **Verification tag** – number that matches a packet to an association. It serves as an identifier for the association and is repeated in every packet during the association
- **Checksum** – 32-bit field contains a CRC-32 checksum

Packets, data chunks, and streams

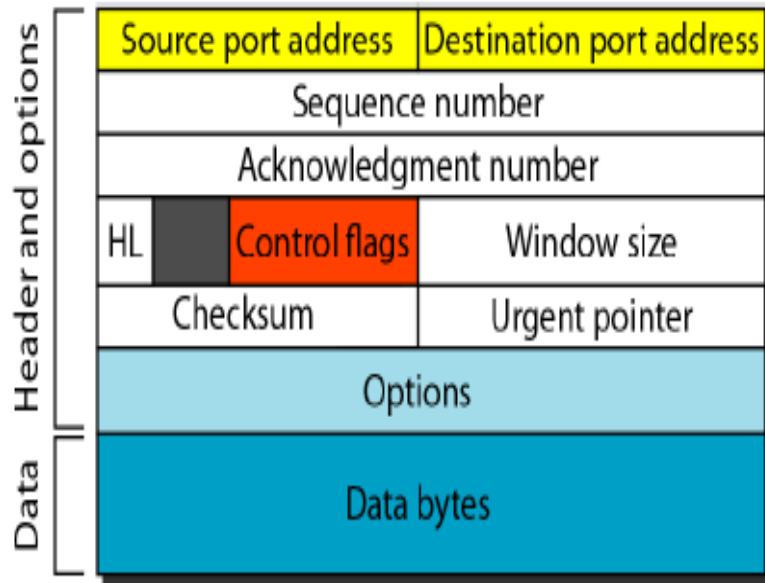


Chunks

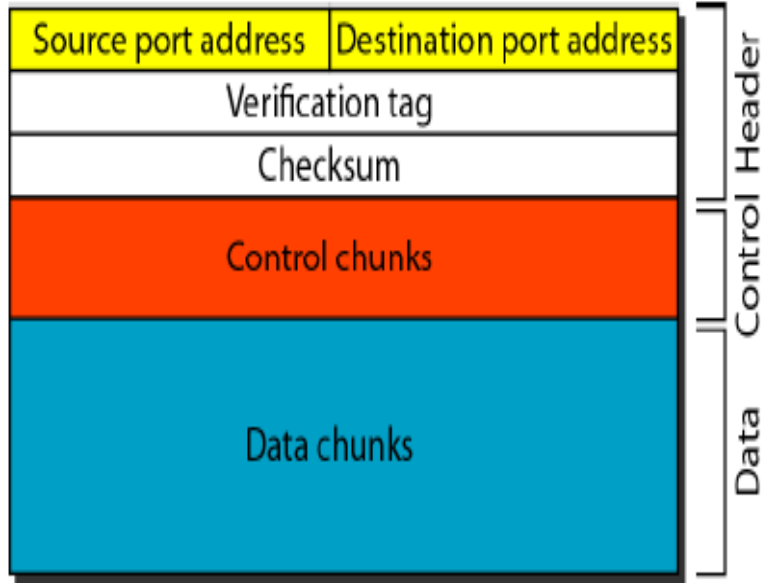


<i>Type</i>	<i>Chunk</i>	<i>Description</i>
0	DATA	User data
1	INIT	Sets up an association
2	INIT ACK	Acknowledges INIT chunk
3	SACK	Selective acknowledgment
4	HEARTBEAT	Probes the peer for liveliness
5	HEARTBEAT ACK	Acknowledges HEARTBEAT chunk
6	ABORT	Aborts an association
7	SHUTDOWN	Terminates an association
8	SHUTDOWN ACK	Acknowledges SHUTDOWN chunk
9	ERROR	Reports errors without shutting down
10	COOKIE ECHO	Third packet in association establishment
11	COOKIE ACK	Acknowledges COOKIE ECHO chunk
14	SHUTDOWN COMPLETE	Third packet in association termination
192	FORWARD TSN	For adjusting cumulative TSN

TCP Segment vs SCTP Segment



A segment in TCP



A packet in SCTP

References



- Forouzan, B.A. 2013. Data Communications and Networking, 5th Ed. McGraw-Hill, New York.